

95–98%

What My Claude Agent Sessions Actually Cost

Real token-burn data from agentic AI workflows.
Not estimates. Not vendor marketing. Ground-truth Anthropic API numbers.

The Setup

The conventional wisdom about AI agent costs is roughly this: it's expensive, context windows are the culprit, and you should probably optimize at some point. Vague, unactionable, and wrong in the specifics.

This report is the result of actually measuring. I built BurnRate — a token-burn observability tool — and ran it against real Cowork sessions and production pipeline data. No estimates, no calibration factors, no extrapolation from toy examples. Ground-truth numbers from the usage fields the Anthropic API embeds in every session.

What I found was worse than the conventional wisdom and more actionable.

How The Data Was Collected

Data sources: four Cowork JSONL session files containing exact per-turn Anthropic API usage fields, plus 33 days of Anthropic console CSV exports from a production n8n pipeline running Claude Haiku 4.5 daily.

The JSONL files contain exact per-turn counts: `input_tokens`, `cache_creation_input_tokens`, `cache_read_input_tokens`, `output_tokens`. No estimation required — these are the numbers Anthropic charges against.

Sessions measured: 186-turn session (4h 7m), 57-turn session (15m), 57-turn session (94m), 124-turn session (21m). Total effective input across all four sessions: 26.3M tokens.

Finding 1

Cache reads are 95–98% of effective input — not 90%

Original hypothesis: re-sent context is ~90% of total token spend by turn 30.

Measured reality: Across 4 Cowork sessions:

Session	Turns	Effective Input	Cache Read Share	Duration
d699f724 (hub)	186	16.2M	98.1%	4h 7m
20162cae (hub)	57	2.3M	88.9%	15m
3a28ed39 (hub)	57	1.7M	95.0%	94m
4b1e8615 (cli)	124	6.0M	96.9%	21m

The 90% estimate was conservative. Real overhead is 95–99%. This is the dominant cost driver by a wide margin — fresh input per turn averages **1–3 tokens** (just the new user message).

Implication: Cutting context window size matters more than any other optimization. A 50% reduction in CLAUDE.md size saves 50% of the largest fixed cost in every session.

Finding 2

Session length is the compounding variable

The 4h 7m session (d699f724) burned 16.2M effective input tokens. The context window grew from ~23K tokens at turn 1 to ~127K by turn 186. Every additional turn added the previous assistant response to the re-sent history.

Cache read tokens per turn (d699f724):

- Turn 1: ~23K
- Turn 50: ~60K
- Turn 100: ~90K
- Turn 186: ~127K

The per-turn cost nearly **6×** between turn 1 and turn 186. A session that runs twice as long costs well over twice as much in effective input.

Action: Use `/compact` at turn 20–30. On a 186-turn session starting at 23K context, compacting at turn 30 would have reduced subsequent per-turn cache reads by an estimated 60–70%, saving ~9M effective input tokens.

Finding 3

Fresh input is negligible — the CLAUDE.md overhead dominates cache-writes

Total fresh input across all 4 sessions: **357 tokens** (literal new user text). Everything else is cache.

The `cache_creation_tokens` (context written to cache on turn 1) ranges from 87K–304K per session. This is the system prompt + CLAUDE.md + project context loaded at session start. It is written to cache once per session but read back on *every subsequent turn*.

The hub CLAUDE.md + project CLAUDE.md combination is estimated at 8,000–15,000 tokens depending on which project is active. Reducing these by 30% would cut cache-write cost on session start and proportionally reduce per-turn cache reads.

Action: Audit all auto-loaded files (CLAUDE.md, STATUS.md, project context). Remove completed roadmap entries, archive old session logs, trim to essential architectural facts only. Target: hub CLAUDE.md under 150 lines, each project CLAUDE.md under 80 lines.

Finding 4

CIPHER API pipeline is already well-optimised

CIPHER's n8n pipeline (measured via Anthropic console CSV, 33 days):

- 86% cache read share on the API key (excellent — means the system prompts are cached and re-used)
- Daily n8n pipeline tokens: 200K–500K/day (manageable)
- Haiku 4.5 is the primary model (correct choice for structured JSON generation)
- 0% cache hit on some Haiku calls (investigation pending — see open questions below)

CIPHER is not a concern for weekly Pro limit since it runs on the API, not through Cowork. It is a concern for API costs, but the 86% cache share means effective cost is ~14% of what naive fresh-input billing would imply.

Finding 5

Project tagging is imperfect — hub sessions show as "other"

Hub-level Cowork sessions (cwd = Cowork Output root, not a project subfolder) were tagged other in v1. Fixed in v2 with cwd suffix detection. After re-ingest, all hub sessions will tag correctly.

Sessions with titles (from /compact or other slash commands) were also ingested with garbled titles. Fixed in v2 with HTML-tag stripping in `_clean_title()`.

What To Do About It

Priority	Action	Estimated Saving
1	Use /compact at turn 25–30 in all long sessions	60–80% of cache_read volume in long sessions
2	Trim all CLAUDE.md files by 30% (archive completed sections)	~30% reduction in cache_creation per session
3	Route short tasks (< 5 turns) to Haiku	~10× less per-token cost for simple lookups
4	Calibrate weekly_token_limit in config.json	Accurate gauge and forecast
5	Ingest daily — don't lose sessions to 30-day window	Complete dataset for trend analysis

What We're Still Measuring

Intellectual honesty requires flagging what the data doesn't yet answer. These are the three open questions the current sample size can't resolve.

- Why do some CIPHER Haiku calls show 0% cache hit? The daily pipeline always starts with the same system prompt — cache should hit after day 1. Possible causes: (a) the Haiku call doesn't reach the cache minimum (1024 tokens?), (b) the cache key is invalidated by dynamic content before the stable suffix, (c) 5-minute cache TTL expiry. Investigate: export a single day's n8n execution log and check the `cache_creation_input_tokens` vs `cache_read_input_tokens` on the Haiku calls.
- What is the real Claude Pro weekly limit? The current config default is 50M tokens. From the session log (cap hit by Wed afternoon, May 6 week), the limit was hit after approximately 2–3 heavy sessions. The 4 sample sessions total 26.3M effective input — if those were in the same week, the limit might be ~30–40M. Calibrate by checking `claude.ai` usage % against a full week of ingested data.
- Are sub-agent spawns double-counted? Some sessions show 3 identical assistant turns at session start (same usage, 1-second apart). These are likely parallel sub-agent spawns (e.g., `TaskCreate` + `TaskUpdate` running concurrently). They ARE separate API calls and DO consume tokens, so counting them is correct. But worth validating against Anthropic console totals.

About

Wilco de Tree is the founder of ZeroNonsense.dev, a Dutch AI advisory practice built around the principle that opinions should be earned through hands-on experiments. BurnRate is one of those experiments: a personal token-burn observability tool built to stop guessing and start measuring.

The findings in this report are from Phase 1 — four sessions of real data, not a model or a guess. More findings will follow as the dataset grows. Buyers of this report receive updates automatically as new data changes the picture.

Questions, data, or disagreements: wilco.de.tree@a-insights.eu · zerononsense.dev